

Headless WordPress + Next.js — Team Standard & Build Process

Status: Living document · **Audience:** WordPress (backend) dev, Frontend (Next.js) dev, Tech Lead **Last reviewed:** June 2026 · **Stack baseline:** Next.js 16 (App Router) · WPGraphQL v2 · WordPress 6.x **Profile:** WordPress-only (no WooCommerce) · Simple products · Free plugins first · Figma-driven UI

This is the reference every project and new team member starts from. It defines *how* we build decoupled WordPress sites, *who owns what*, and the non-negotiables for SEO and security. Conventions here are defaults — deviate only with a documented reason.

0. Locked decisions for this profile

These were decided up front. They shape every plugin and schema choice below.

Decision	Choice	Implication
Commerce engine	None — no WooCommerce	Products are content, not transactions. No cart/checkout/payments/stock engine.
Products	Custom Post Type product	Modelled like any other content type.
Variants (colour/size/shape)	Native taxonomies by default (see §5)	Free, native, filterable, first-class in GraphQL. No repeater plugin needed for simple products.
Custom fields	Free solutions only (Gutenberg blocks + a free field plugin/code)	No ACF Pro. Paid options are listed <i>separately</i> in §13 if you choose to buy.
Flexible "page builder" content (e.g. landscape page)	Native Gutenberg block editor + WPGraphQL Content Blocks	Editors compose pages from blocks; frontend renders blocks as React components. Free.
Design source of truth	Figma	Tokens + components flow Figma → Tailwind → React (see §7).

If any of these change later (e.g. you add real transactions), revisit §0 first — the rest of the document follows from it.

1. Plugin philosophy: free-first

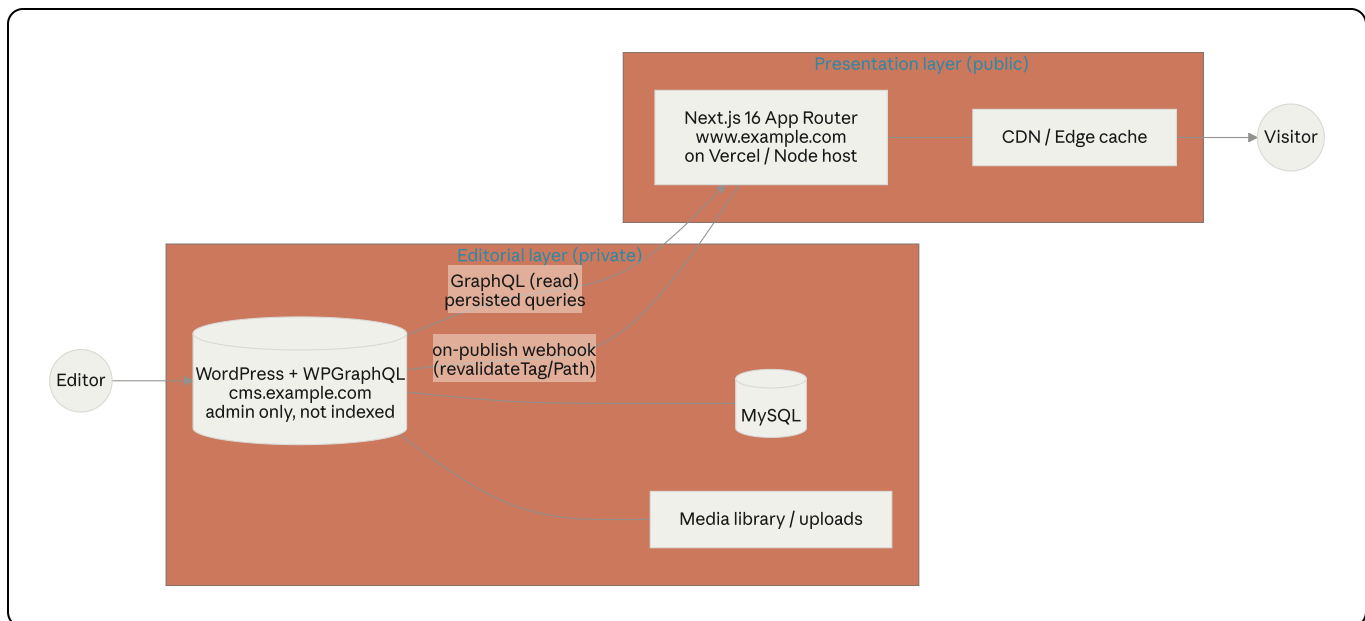
Everything required to ship this site is achievable on **100% free, open-source plugins**. We use paid plugins only when they buy a clear, worth-it advantage, and we decide that explicitly — never by default. All paid options are quarantined in **§13 (Optional paid upgrades)** so the free baseline stays obvious.

Two rules:

1. **Prefer native WordPress + free, well-maintained extensions** over premium plugins.
2. **Prefer modelling in code** (registered in a small must-use plugin) over click-configured settings, so the content model is version-controlled and reproducible across environments.

2. Architecture overview

WordPress is the editorial backend only. Visitors never hit WordPress directly. Next.js fetches content over GraphQL and renders everything.



Three layers:

1. **WordPress backend** — managed host, *frontend theme disabled*, serves the admin UI + GraphQL API only. Main attack surface, so it's locked down (§10).
2. **Data/API layer** — WPGraphQL exposes the whole content model (posts, the **product** CPT, taxonomies, custom fields, Gutenberg blocks, menus, SEO fields) as one typed GraphQL schema.
3. **Next.js frontend** — consumes GraphQL; hybrid rendering (static where possible, ISR for content that changes, SSR/client only where genuinely dynamic).

The contract between the two developers

The **GraphQL schema is the interface** between backend and frontend. Both devs agree the content model first (§5), then work in parallel:

- **WordPress dev** owns the schema's *existence and shape* (CPT, taxonomies, fields, blocks, exposing them correctly in GraphQL).
- **Frontend dev** owns *consumption* (typed queries, rendering, routing, caching) and the UI built from Figma.
- Neither silently renames or reshapes a field — that breaks the frontend silently (missing field) or loudly (type error). **Automated tests asserting on GraphQL response shape are mandatory.**

To keep the frontend dev unblocked: agree the schema up front, generate types from it (GraphQL Code Generator), and use the WPGraphQL IDE / introspection so the frontend builds against the real schema as fields land.

3. The free stack (decided defaults)

WordPress side — owned by the WordPress dev (all free)

Plugin / approach	Purpose	Notes
WordPress 6.x	CMS / content store	Managed host.
WPGraphQL (v2)	The GraphQL API. The lynchpin.	Free. Persisted queries + CDN caching in v2.
WPGraphQL Content Blocks (WP Engine)	Exposes Gutenberg blocks as structured <code>editorBlocks</code> data	Free, actively maintained. Powers flexible pages (home sections, landscape page). No config after install.
Custom fields — pick one (all free): Carbon Fields (code-defined) · Meta Box core (UI) · Pods (UI) · or native <code>register_meta</code> + <code>register_graphql_field</code>	Structured fields on <code>product</code> and elsewhere	See §5 for the recommendation. Each has a free WPGraphQL bridge (Carbon Fields and Meta Box GraphQL extensions are free; native needs no bridge).

Plugin / approach	Purpose	Notes
Custom Post Type UI + its WPGraphQL setting <i>(or register CPTs in code — preferred)</i>	Register the <code>product</code> CPT + taxonomies	We prefer a must-use plugin in code so it's version-controlled.
Yoast SEO (free) + WPGraphQL Yoast SEO Addon (free)	SEO metadata, sitemaps source, schema. Exposed in GraphQL.	Equally valid free alternatives: SEOPress (free) + its WPGraphQL addon , or Rank Math (free, headless support) . Pick one.
Forms — native Next.js form (default, §6) <i>or</i> Ninja Forms (free) + WPGraphQL Ninja Forms extension (free)	Contact form	Gravity Forms is paid → §13.
Redirection (free)	Manage 301/302 redirects	Frontend reads these and applies them (§8).
WPGraphQL JWT Authentication (free)	Token auth for any writes / draft previews	Only needed for authenticated operations.
Wordfence <i>or</i> Solid Security (free tiers)	Hardening, firewall, 2FA, login limiting	§10.

Frontend side — owned by the Frontend dev

Tool	Purpose	Notes
Next.js 16 (App Router)	The application	App Router is the only sensible default; Pages Router is maintenance-only. Turbopack is the default bundler.
React 19.2	UI	Ships with Next 16; React Compiler available.
TypeScript	Type safety end-to-end	Non-negotiable with a typed GraphQL schema.
graphql-request (lean) <i>or</i> Apollo / TanStack Query	GraphQL fetching	Prefer lightweight <code>graphql-request</code> /native <code>fetch</code> in Server Components. Reserve a client cache for genuinely client-side data (live filters).

Tool	Purpose	Notes
GraphQL Code Generator	Generates TS types from the schema	Keeps frontend and backend in lockstep.
Tailwind CSS	Styling	Design tokens come from Figma (§7).
Native Metadata API (<code>generateMetadata</code>)	SEO tags	Prefer over <code>next-seo</code> in App Router; JSON-LD via a helper.
Cloudflare Turnstile (free) or reCAPTCHA	Form spam protection	Verified server-side.
Zod	Runtime validation at trust boundaries	Forms, API inputs.
Vercel (or Node/Cloudflare host)	Hosting + CDN + on-demand ISR	Path of least resistance for Next 16.

On Faust.js: WP Engine's framework bundles auth, preview, and WP-style routing, but moves slowly and ships opinions that can fight Next 16 App Router patterns. **Default is plain Next.js + WPGraphQL + a thin preview layer (§9).** Reach for Faust only if you want the framework to make those decisions for you.

4. Project / repo structure

Two repos: WordPress backend (must-use plugins, field/CPT code, config as code) and the Next.js frontend. They deploy independently.

```

frontend/
├─ app/
│  ├─ (marketing)/
│  │  ├─ page.tsx           # Home (Gutenberg blocks → components)
│  │  ├─ contact/page.tsx  # Contact
│  │  └─ [slug]/page.tsx   # Dynamic CMS pages (e.g. "landscape")
│  ├─ products/
│  │  ├─ page.tsx          # Product list (+ taxonomy filters)
│  │  └─ [slug]/page.tsx   # Product detail (variants)
│  ├─ blog/
│  │  ├─ page.tsx          # Blog listing (paginated)
│  │  └─ [slug]/page.tsx   # Blog detail
│  ├─ sitemap.ts           # Generated sitemap(s)
│  ├─ robots.ts           # robots rules
│  └─ api/
│     ├─ revalidate/route.ts # On-publish webhook target
│     └─ contact/route.ts   # Form handler (or a Server Action)
├─ lib/
│  ├─ graphql/             # client + queries
│  ├─ blocks/              # Gutenberg block → React component map
│  ├─ seo/                 # metadata + JSON-LD builders
│  └─ wp/                  # mappers (WP shape → app shape)
├─ components/             # Built from Figma components (§7)
├─ styles/tokens.css       # Design tokens (mirrors Figma)
├─ tailwind.config.ts      # Tokens wired into Tailwind theme
├─ generated/              # GraphQL Codegen output (types)
└─ codegen.ts

backend/
├─ mu-plugins/             # CPT + taxonomies + GraphQL tweaks
(versioned)
├─ fields/                 # Carbon Fields definitions (if used) –
code = versioned
├─ blocks/                 # Custom Gutenberg blocks (block.json) if
needed
└─ composer.json           # Plugins pinned by version where possible

```

Version-control the content model. CPT/taxonomies → must-use plugin. Field definitions → code (Carbon Fields), or export Meta Box/Pods config to the repo. Custom blocks → `block.json` in the repo. The model is reproducible and reviewable in PRs, never click-configured in production.

5. Content modelling (do this first, together)

The single most important shared artifact. Agree it before writing queries, fields, or components.

Pages & their data sources

Page	Source in WP	GraphQL root
Home	"Home" Page built with Gutenberg blocks	<code>page(id:"/", idType: URI)</code> → <code>editorBlocks</code>
Product list	<code>product</code> CPT + taxonomies	<code>products(where:...)</code>
Product detail	A single <code>product</code>	<code>product(id:..., idType: SLUG)</code>
Contact	"Contact" Page (intro copy) + the form	<code>page</code> (+ form source if using Ninja Forms)
Blog listing	Core <code>posts</code> + categories/tags	<code>posts(first:..., after:...)</code>
Blog detail	A single <code>post</code>	<code>post(id:..., idType: SLUG)</code>
Landscape CMS page	<code>page</code> (or <code>landscape</code> CPT) built with Gutenberg blocks	<code>page</code> / <code>landscape</code> → <code>editorBlocks</code>

Product variant model — two free options

Option A — Native taxonomies (default for simple products). Register `colour`, `size`, `shape` as custom taxonomies. Tag each product with its available terms. This is free, native, **first-class in WPGraphQL**, and makes the product list **filterable** out of the box. Best when a variant is just a *selectable attribute* and doesn't carry its own price/stock/image.

php

```
// mu-plugin (illustrative)
register_taxonomy('colour', 'product', [
    'show_in_graphql' => true,
    'graphql_single_name' => 'colour',
    'graphql_plural_name' => 'colours',
    'hierarchical' => false,
]);
// repeat for 'size' and 'shape'
register_post_type('product', [
    'show_in_graphql' => true,
    'graphql_single_name' => 'product',
    'graphql_plural_name' => 'products',
    'public' => true,
    'has_archive' => false,
    'supports' => ['title', 'editor', 'thumbnail', 'excerpt'],
]);
```

Frontend: render selectors from the product's `colours`/`sizes`/`shapes`; reflect the choice in the UI (and query params for shareable state).

Option B — Per-combination data (only if needed). If a specific combination needs its own image/SKU/short description (e.g. show the red photo when "red" is selected), model variants as either:

- a `product_variant` **child CPT** (each linked to its parent product), exposed in GraphQL; or
- a **Carbon Fields "Complex" field** (free repeater-equivalent) on the product, exposed via the free WPGraphQL Carbon Fields extension.

Start with Option A. Only adopt Option B if the design genuinely requires per-variant assets. Verify the GraphQL output of any complex/nested field early — confirm the schema shape before building the frontend against it.

"Landscape" dynamic CMS page — Gutenberg blocks, not ACF

The flexible, editor-driven page (dynamic image + content) uses the **native block editor as the page builder**, with **WPGraphQL Content Blocks** exposing blocks as structured data. The frontend maps each block to a React component.

```
landscape page → editorBlocks:
  core/cover or core/image    → <HeroImage>
  core/heading                → <Heading>
  core/paragraph              → <RichText>
  core/columns + core/column  → <TwoColumn>
  core/gallery                → <Gallery>
  core/buttons                → <CtaButtons>
  (custom) acme/hero-cta      → <HeroCta>      # register a block.json only if
core blocks don't cover the design
```

How extension works:

- **Core blocks** (image, cover, heading, paragraph, columns, gallery, buttons) cover most "image + content" layouts with **zero custom code**.
- **Need a bespoke section** (e.g. a specific hero with overlay + CTA)? Register a **custom block** (`block.json`, free) so editors get a designed, constrained block. One block (backend) + one component + one entry in the block→component map (frontend).
- The set of available blocks should mirror the **designed section components in Figma** (§7), so editors compose pages from a known, on-brand kit.

This same pattern powers the **Home page's** flexible sections — it generalises to any editor-composed page.

Rendering WordPress HTML safely

Block `renderedHtml` and classic content come through as HTML. **Sanitise** (allowlist) before rendering, and prefer mapping blocks to components where you need control. Never inject unsanitised HTML from any field a user could influence.

6. Page-by-page build spec

Rendering key: **SSG** (build-time static), **ISR** (static + revalidate), **SSR** (per-request), **CSR** (client island).

Page	Strategy	Routing	Key notes
Home	ISR	<code>app/(marketing)/page.tsx</code>	Render <code>editorBlocks</code> via the <code>block→component map</code> . Optimise LCP (hero via <code>next/image</code>). Revalidate on publish.
Product list	ISR (or SSR if filtering server-side)	<code>app/products/page.tsx</code>	Filter by <code>colour</code> / <code>size</code> / <code>shape</code> taxonomies; cursor pagination from GraphQL. Filters can hydrate client-side over static shell.
Product detail	ISR + <code>generateStaticParams</code>	<code>app/products/[slug]/page.tsx</code>	Variant selectors from taxonomies (Option A) variant data (Option B) One canonical URL per product.
Contact	Static page + interactive form island	<code>app/contact/page.tsx</code>	Copy is static/ISR; form a Client Component → Server Action/Route Handler with Turnstile Zod (§ below).
Blog listing	ISR + <code>generateStaticParams</code> for pages	<code>app/blog/page.tsx</code> (+ pagination)	Paginate (<code>/blog?page=2</code> or <code>/blog/page/[n]</code>). Revalidate on publish.
Blog detail	SSG/ISR	<code>app/blog/[slug]/page.tsx</code>	<code>generateStaticParams</code> from all slugs; ISR for edits. Render content safely.

Page	Strategy	Routing	Key notes
Landscape CMS page	ISR	<code>app/[slug]/page.tsx</code>	Renders <code>editorBlock</code> New <i>content</i> needs no redeploy; only a new <i>block type</i> does.

Variant handling on the product detail page

1. Fetch the product with its taxonomy terms (Option A) or variant entries (Option B).
2. Render selectors for **colour / size / shape**; disable combinations that don't exist.
3. Update displayed info on selection (image/short description where Option B provides it).
4. **SEO URL strategy: keep one indexable URL per product** (`/products/red-widget`). Reflect the selected variant in **query params** (`?colour=red&size=m`) for shareable state, with the **canonical set to the base product URL**. Don't create a separately indexable URL per variant (duplicate content). Exception: a variant with genuinely distinct demand gets its own `product` entry.
5. Emit **Product** structured data (§9).

Contact form (free, default = native Next.js)

Simplest free path, no form plugin: a Client Component form → **Server Action / Route Handler** that (a) validates with **Zod**, (b) verifies **Cloudflare Turnstile** server-side, (c) sends email (Resend free tier or SMTP), and optionally (d) stores the submission. Keep all secrets server-side; rate-limit the endpoint; include a honeypot.

*Alternative: **Ninja Forms (free)** managed in WP + its **free WPGraphQL extension** for GraphQL submission — choose this if editors must manage form fields/notifications and store entries in WordPress.*

7. Design-to-development workflow (Figma)

Figma is the **single source of truth for UI**. The flow is Figma → tokens → Tailwind → components → pages. Do this *before* assembling pages so the design system, not ad-hoc CSS, drives the build.

a) Extract design tokens first

Pull the primitives out of Figma and encode them once so code mirrors design:

- **Colours, typography scale** (family, sizes, weights, line-heights), **spacing scale, breakpoints, radii, shadows.**
- Encode in `styles/tokens.css` (CSS variables) and wire into `tailwind.config.ts` `theme.extend`. Components then use token-based utilities, never hard-coded hex/px.
- Result: a colour or spacing change in Figma is a token change in one place, not a hunt through components.

b) Component-driven build

- Use **Figma Dev Mode** for measurements, spacing, CSS hints, and component status.
- Build a **component library first** (buttons, inputs, cards, nav, product card, the Gutenberg block components), each matching a Figma component **1:1**. Keep a **Figma-component → React-component mapping doc**.
- Assemble pages from those components. This keeps the UI consistent and makes the Gutenberg block kit (§5) line up with designed sections.

c) Assets

- **Icons / illustrations / decorative chrome:** export from Figma as **SVG**, optimise (SVGO), commit to the repo. Prefer inline SVG for icons.
- **Content imagery** (product photos, blog images): comes from **WP media** via `next/image` (sized, lazy, modern formats), not Figma.
- Export raster assets at the densities the design needs; define responsive behaviour from Figma breakpoints.

d) Handoff ritual & quality gates

- **Design sign-off before build** for each template. If a design can't be built cleanly, resolve it **in Figma**, don't improvise in code.
- Frontend dev fills gaps designs often miss: **focus states, hover/active, empty/error/loading states, contrast, semantic structure, keyboard nav**. Check colour contrast against WCAG.
- Review built pages **against the Figma frames** (spacing, type, states) before QA.

8. Redirects & migration hygiene (if replacing an existing site)

- Export every existing URL; map 1:1 to new routes; 301 anything that changes.
- Keep permalink structure identical where feasible (`/blog/post-slug`, `/products/slug`).

- Read the **Redirection** plugin's rules and apply them in `next.config` `redirects()` or `proxy.ts` (formerly middleware).
- Submit the new sitemap in Search Console; monitor coverage + 404s for several weeks.
- Validate structured data (Rich Results Test) and metadata on a sample of each page type before go-live.

9. SEO strategy (the part that makes or breaks headless)

In headless, SEO is *your* job in the frontend — WordPress no longer renders the `<head>`. The plugin (Yoast/SEOPress/Rank Math, free) is the **source of SEO data**; Next.js renders it.

a) Per-page metadata

Query the SEO addon's fields and feed `generateMetadata`:

```
ts
// app/blog/[slug]/page.tsx
export async function generateMetadata({ params }): Promise<Metadata> {
  const { post } = await getPost(params.slug); // includes seo { title, metaD
  return {
    title: post.seo.title,
    description: post.seo.metaDesc,
    alternates: { canonical: post.seo.canonical }, // always the PUBLIC dom
    openGraph: { title: post.seo.opengraphTitle, images: [post.seo.opengraphIma
    twitter: { card: 'summary_large_image' },
    robots: { index: post.seo.metaRobotsNoindex !== 'noindex', follow: true },
  };
}
```

Do this for **every** route type so editors keep control of titles/descriptions/canonicals.

b) Structured data (JSON-LD) per page type

- **Organization / WebSite** — site-wide.
- **BreadcrumbList** — product, blog detail, CMS pages.
- **Product** — product detail (name, image, description; add `Offer` only if you show price/availability).
- **Article / BlogPosting** — blog detail (author, dates, image).
- Keep one builder per type in `lib/seo/jsonld.ts`; inject via `<script type="application/ld+json">`.

c) Sitemaps (classic headless gotcha)

The WP plugin's sitemap points at the **wrong (WP) domain**. Generate the sitemap in **Next.js** (`app/sitemap.ts`) by querying all published slugs (pages, products, posts, CMS pages) and emitting correct public URLs. Split into multiple sitemaps + an index if large; set `lastModified` from WP `modified` dates.

d) robots, canonicals, redirects

- `app/robots.ts` — allow the public site, **disallow/secure the WP backend domain**, link the Next.js sitemap.
- Canonicals always point at the public domain, never `cms.example.com`.
- Apply redirects from the Redirection plugin (§8). Preserve old URL structure on migration to keep rankings.

e) Performance = SEO (Core Web Vitals)

- `next/image` for all content media (WP media URLs as remote patterns).
- Static/ISR by default; minimise client JS (Server Components; islands only where needed).
- Watch **LCP** (hero images), **CLS** (reserve dimensions), **INP** (small client bundles). A key reason to go headless is shedding plugin-bloated WP JS — don't reintroduce it.

f) Draft preview for editors (don't skip)

Editors must preview unpublished content. Implement with **Next.js Draft Mode**: a preview route validates a token, enables draft mode, and the GraphQL query fetches the draft (authenticated via WPGraphQL JWT). This is the main thing Faust gives for free — if you go plain Next.js, build this thin layer deliberately.

10. Security measures (WordPress is the attack surface)

Headless *reduces* surface area (no public theme/frontend plugin exploits) but the **admin + API remain exposed**. Lock both down.

WordPress backend hardening

- **Disable the public frontend / themes**. Redirect all non-admin, non-API frontend requests to the Next.js site.
- **Obscure & isolate the backend**. Separate subdomain (`cms.example.com`), kept out of search indexes, with `wp-admin`/`wp-login.php` behind an **IP allowlist**, **HTTP basic**

auth, or VPN/Cloudflare Access.

- **2FA** on all admin accounts; **limit login attempts**; lock/rename the login endpoint.
- **Least-privilege roles** — editors get editor rights, not admin. Remove unused accounts.
- **Disable XML-RPC** unless specifically needed (brute-force/DDoS vector).
- **Disable the file editor** (`(DISALLOW_FILE_EDIT)`).
- **Rotate security keys/salts**; enforce strong passwords.
- **Keep core + plugins + themes updated** — outdated plugins are the #1 WordPress compromise vector. Pin versions in `(composer.json)`; update deliberately; subscribe to advisories.
- **WAF + CDN** in front of the backend; **rate-limit the GraphQL endpoint**.
- **Automated backups** (DB + uploads) with a tested restore.

API security

- **CORS**: restrict the GraphQL endpoint to known frontend origins + the build system, not `(*)`.
- **Auth for writes**: public *reads* need no auth. The moment you **write** (form submissions) or **read drafts** (preview), authenticate — **WPGraphQL JWT** server-side only. Never expose write credentials to the browser.
- **Disable GraphQL introspection in production** (or restrict it); enable **query depth/complexity limits + persisted queries** to block abusive/expensive queries.
- **Rate-limit** GraphQL and any form endpoints.

Frontend / Next.js security

- **Keep Next.js patched**. Next.js has shipped multiple high/critical advisories (RSC, middleware/proxy bypass, SSRF, cache poisoning, image-optimization DoS). Track the security feed and **upgrade promptly** on a cadence.
- **Secrets in env vars**, never in the repo. Only `(NEXT_PUBLIC_*)` is truly public; JWT/write creds are **server-only**.
- **Content Security Policy** (nonce-based) + security headers (HSTS, X-Content-Type-Options, Referrer-Policy).
- **Sanitise all WordPress HTML** before rendering (allowlist). Treat any user-influenced field as untrusted.
- **Form protection**: Turnstile/reCAPTCHA **verified server-side**, honeypot, Zod validation, rate limiting.
- **Dependency scanning**: `(npm audit)` + Dependabot/Renovate on both repos; merge security PRs quickly.
- **HTTPS everywhere**, including the backend ↔ frontend hop.

Security checklist (gate before launch)

- ☐ WP public frontend disabled / redirected
 - ☐ Backend on isolated subdomain, noindexed, admin access restricted (IP/basic-auth/VPN)
 - ☐ 2FA + login limiting on all accounts; least-privilege roles
 - ☐ XML-RPC + file editor disabled; salts rotated
 - ☐ Core/plugins/themes current; update cadence agreed
 - ☐ CORS locked to known origins; introspection off in prod; depth/complexity limits on
 - ☐ Writes authenticated **server-side only**; no secrets in client bundle
 - ☐ Next.js on a patched version; CSP + security headers set
 - ☐ Form spam protection verified server-side; inputs validated
 - ☐ WAF/CDN active; rate limits on API + forms
 - ☐ Backups running + a restore tested
 - ☐ `npm audit`/Dependabot clean on both repos
-

11. Environments, Git & collaboration workflow

Environments

Env	WordPress	Frontend	Purpose
Local	Local WP (Docker/LocalWP) or shared dev backend	<code>next dev</code>	Day-to-day dev. Frontend can point at the shared staging backend to skip local WP setup.
Staging	Staging backend	Preview deploy	Content + feature QA, design review against Figma, SEO/structured-data validation.
Production	Production backend (private)	Production deploy	Live.

Git workflow

- **Trunk-based with short-lived feature branches** → PR → review → merge to `main`.
- Every PR: typecheck + lint + the **schema-shape tests** must pass in CI.
- **Conventional commits**; PRs touching fields/blocks reference the content-model doc.
- Backend PRs include the CPT/taxonomy/field/block code so the model change is reviewable.

How the two devs stay unblocked

1. **Day 1:** agree the content model (§5) together — the contract.
 2. **WP dev** builds the CPT/taxonomies/fields/blocks + exposes them in GraphQL.
 3. **Frontend dev** runs **GraphQL Codegen** → typed queries/components; builds against real fields as they land (mock where a field isn't ready), assembling UI from the Figma component library (§7).
 4. **Schema changes are a shared event** — announced, reviewed, reflected in regenerated types + updated tests. No silent renames.
-

12. Deployment, caching & content freshness

Deploy

- **Frontend** → **Vercel** (or Node/Cloudflare). PRs get preview URLs; `main` auto-deploys to production.
- **Backend** → **managed WP host**. Deploys independent of the frontend.

Caching (Next.js 16 specifics)

Next 16 caching is **explicit** — `fetch` is *not* cached by default. Use `revalidate`/`cacheLife` + `cacheTag` deliberately. Tag pages (e.g. `post:slug`, `product:slug`, `page:slug`). WPGraphQL persisted queries + the CDN cache reduce backend load.

On-publish freshness (the important bit)

When an editor publishes/updates, the static page must refresh **without a full rebuild**:

```
Editor publishes in WP
  | (save_post / transition_post_status / WPGraphQL Smart Cache)
  ▼
POST https://www.example.com/api/revalidate (with a shared secret)
  |
  ▼
Next.js Route Handler verifies secret → revalidateTag('product:slug') (or
revalidatePath)
  |
  ▼
Affected page regenerates on next request; visitors see fresh content
```

Implement `app/api/revalidate/route.ts`, protect with a shared secret, and wire a WordPress hook (on `save_post`/`transition_post_status`), or WPGraphQL Smart Cache) to call it.

13. Optional paid upgrades (only if you choose to buy)

The build above is fully free. These are **optional** purchases, each with a clear reason — adopt only if the trade is worth it.

Paid option	~Cost	What it buys you vs the free path	Worth it when...
ACF Pro + WPGraphQL for ACF	~\$59/yr (per-site/agency tiers)	The most polished field UI, mature repeater + flexible-content fields, and the best-supported field→GraphQL bridge in headless. Smoother than wiring Carbon Fields/Meta Box yourself.	Non-devs must manage complex fields via UI, <i>and</i> you want the most battle-tested headless field tooling.
Gravity Forms + its WPGraphQL extension	~\$59-259/yr	Robust forms with conditional logic, multi-step, entry management, notifications, and the strongest WPGraphQL form extension.	Forms get complex, or marketing needs to build/manage forms and review entries in WP.
Meta Box AIO / premium extensions	varies	Adds Meta Box's premium field types (e.g. certain cloneable/group setups) if you've standardised on Meta Box and hit a free-tier limit.	You chose Meta Box and need a premium field its free core lacks.
WPML (or Polylang Pro)	~\$99-199/yr	Mature multilingual content + translation management, exposed via WPGraphQL.	The site must be multilingual.
Managed headless hosting (WP Engine Atlas / Kinsta)	from ~\$49/mo	Bundled WP + Node hosting, CDN, and headless-specific tooling/preview from one vendor.	You want a single vendor and less infra to run yourself.

Recommendation order if budget opens up: **Gravity Forms** (if forms grow) → **ACF Pro** (if field management becomes painful) → the rest only as the specific need (multilingual, hosting consolidation) appears.

14. Standard operating procedures (reusable playbooks)

Adding a new block / section type

1. **Design:** add/approve the section component in Figma (§7).
2. **Backend:** if core blocks don't cover it, register a custom block (`block.json`); confirm it appears in `editorBlocks` via WPGraphQL.
3. **Frontend:** run Codegen → types; add the block→component mapping + the component (matching Figma).
4. Extend the **shape test** for the new block/fields.

Adding a new content type / field

1. **Backend:** register the CPT/taxonomy/field in code; expose in GraphQL; PR.
2. **Frontend:** Codegen → query → render; PR.
3. Update tests + the content-model doc.

Pre-launch checklist

- ☐ No WooCommerce; `product` CPT + variant taxonomies (or Option B) implemented
- ☐ All 8 page types built with correct rendering strategies
- ☐ Gutenberg block→component map covers the designed section kit
- ☐ Design reviewed against Figma; tokens wired into Tailwind; states (empty/error/loading/focus) handled
- ☐ `generateMetadata` on every route; canonicals → public domain
- ☐ JSON-LD per page type; validated in Rich Results Test
- ☐ Sitemap generated in Next.js with correct URLs; submitted to Search Console
- ☐ `robots.ts` allows public site, secures backend
- ☐ Redirects mapped (esp. migration); 404s monitored
- ☐ Draft preview works for editors
- ☐ Core Web Vitals checked (LCP/CLS/INP) on key templates
- ☐ On-publish revalidation webhook working end-to-end
- ☐ Full **security checklist** (§10) green
- ☐ Backups + restore tested
- ☐ Schema-shape tests + lint + typecheck passing in CI

Code review standards

- No `any` for GraphQL data — use generated types.

- No unsanitised WP HTML rendered.
 - No secrets/write-credentials in client components or `NEXT_PUBLIC_*`.
 - Every new route type ships metadata + (where relevant) JSON-LD.
 - Components use design tokens, not hard-coded values.
 - Schema/field/block changes link the content-model doc and update tests.
-

15. Quick reference — recommended defaults at a glance

- **Frontend:** Next.js 16 App Router + TypeScript + Tailwind, on Vercel.
- **Data:** WPGraphQL v2, fetched in Server Components via `graphql-request`, types via GraphQL Codegen.
- **Products:** `product` CPT; variants via **native** `colour`/`size`/`shape` **taxonomies** (Option A); per-combination data via child CPT or Carbon Fields only if needed (Option B). **No WooCommerce.**
- **Flexible pages (home, landscape):** native **Gutenberg blocks** + **WPGraphQL Content Blocks** → React component map.
- **Custom fields:** free — **Carbon Fields** (code) / **Meta Box** or **Pods** (UI) / native meta. No ACF Pro.
- **SEO:** **Yoast free** + **WPGraphQL Yoast addon** (or SEOPress/Rank Math free); metadata via `generateMetadata`; sitemap + robots generated in Next.js; JSON-LD per page type.
- **Forms:** **native Next.js Server Action** + **Turnstile** + **Zod** (default), or **Ninja Forms free** + **GraphQL extension**.
- **Design:** Figma → tokens → Tailwind → component library → pages; block kit mirrors designed sections.
- **Freshness:** ISR + on-publish webhook → `revalidateTag`.
- **Security:** private/locked backend, server-side-only writes, CORS + query limits, patched Next.js, CSP, scanned deps.
- **Paid (optional, §13):** Gravity Forms → ACF Pro → others, only as a specific need appears.

Versions move fast. Next.js ships frequent security patches; WordPress/plugins update constantly. Re-confirm the current stable Next.js version and check the Next.js + WordPress/plugin security feeds at the start of each project and on a regular patch cadence.